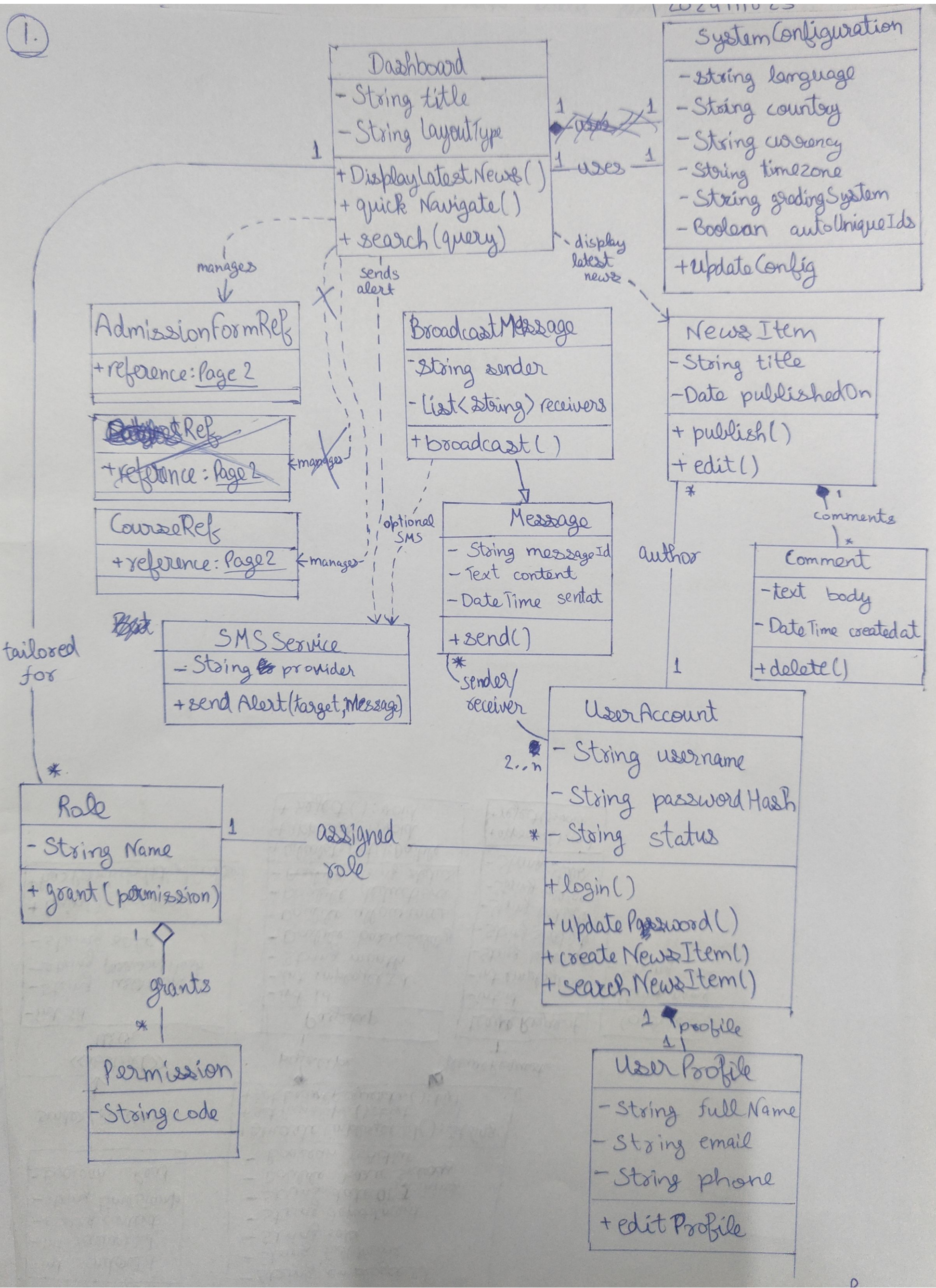
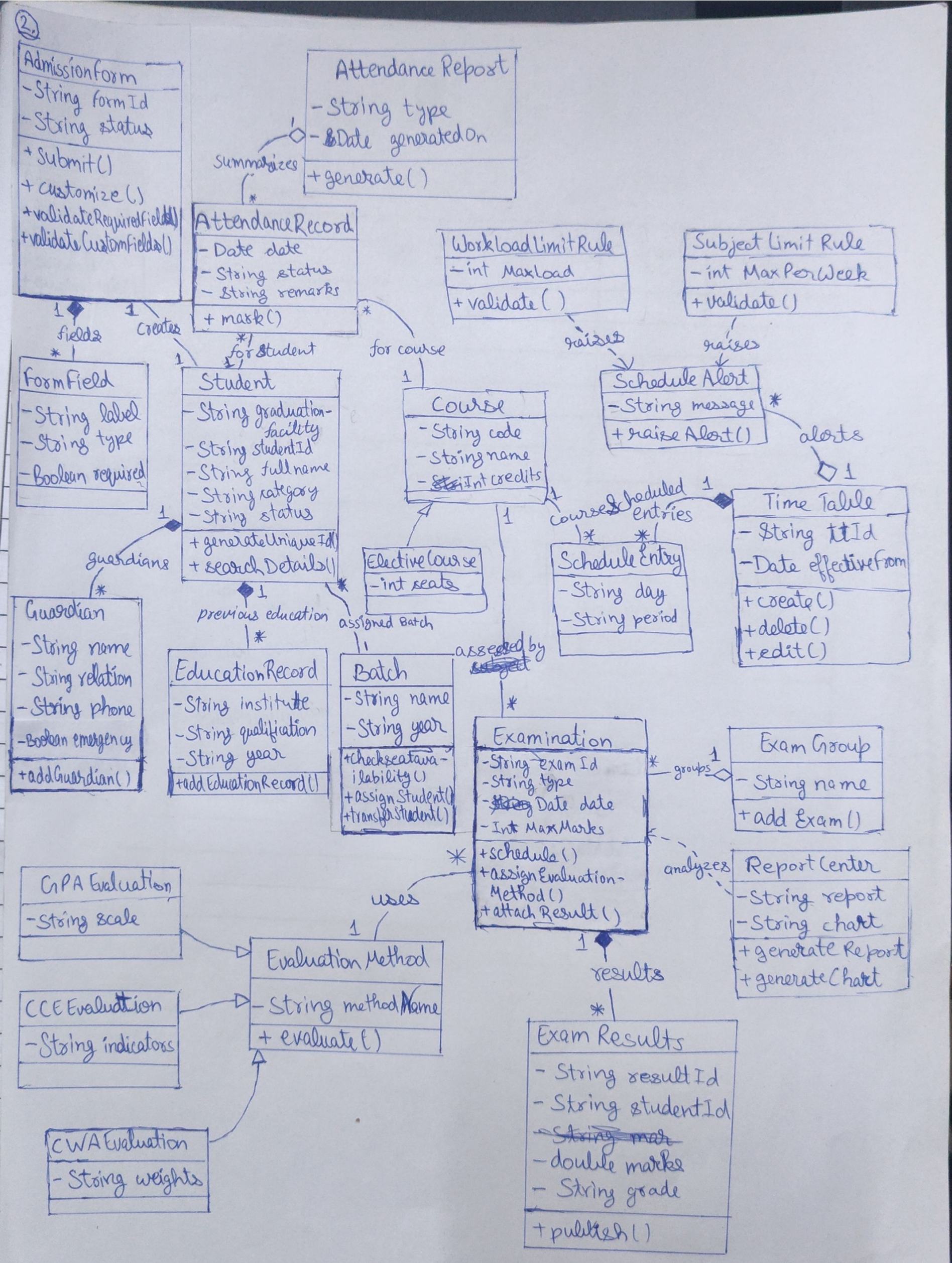
A3 Report

2024111023, 2024117009

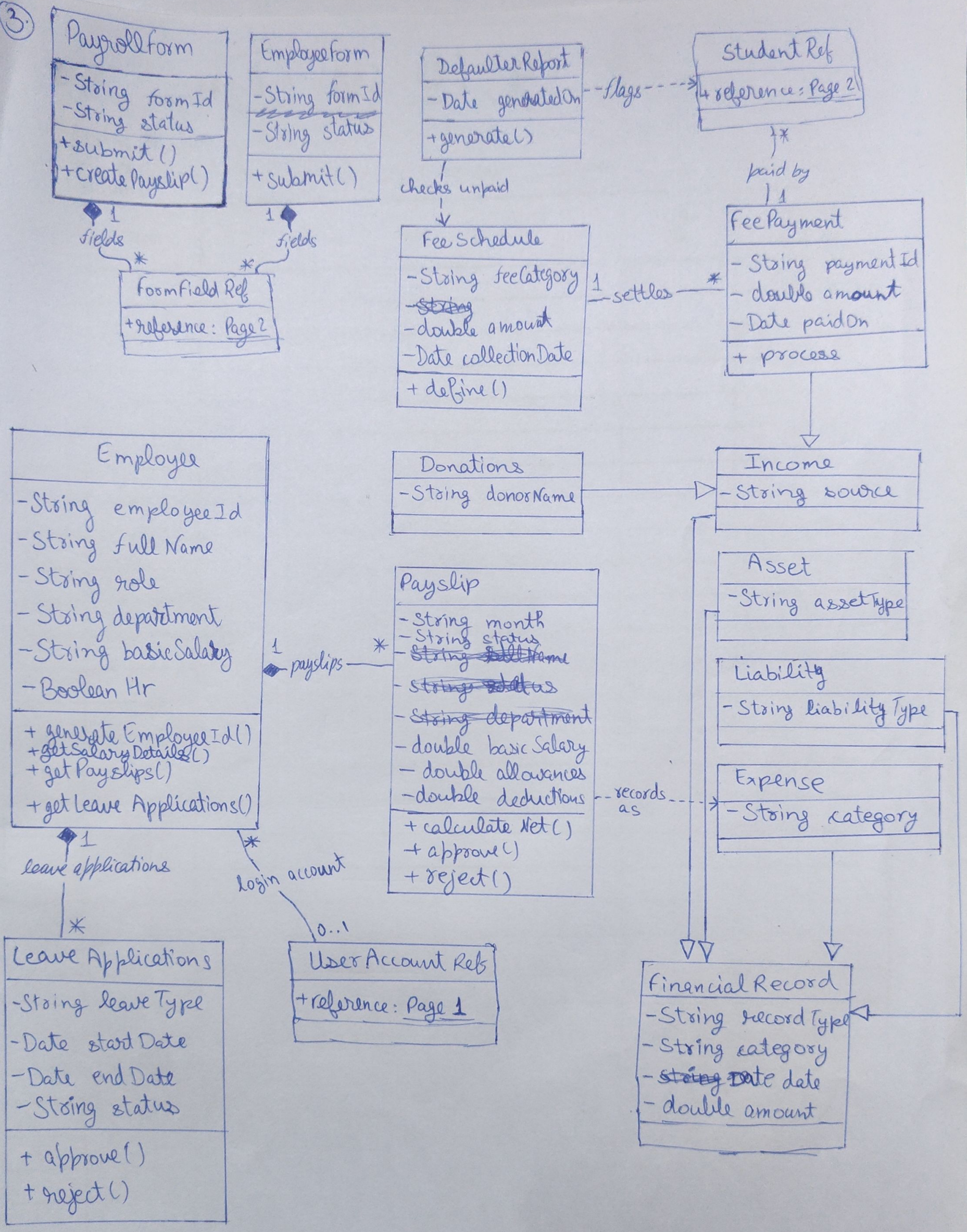
Task 1.1: UML



2.


AdmissionForm
- String formId
- String status
+ Submit()
+ customize()
+ validateRequiredFields()
+ validateCustomFields()
Attendance Report
- String type
- Date generatedOn
+ generate()
summarizes
AttendanceRecord
- Date date
- String status
- String remarks
+ mark()
WorkloadLimitRule
- int MaxLoad
+ validate()
Subject Limit Rule
- int MaxPerWeek
+ validate()
fields
creates
for student
for course
raises
raises
FormField
- String label
- String type
- Boolean required
Student
- String graduation-facility
- String studentId
- String fullname
- String category
- String status
+ generateUniqueId()
+ searchDetails()
Course
- String code
- String name
- Int credits
Schedule Alert
- String message
+ raiseAlert()
alerts
Time Table
- String ttId
- Date effectiveFrom
+ create()
+ delete()
+ edit()
scheduled entries
course
Schedule Entry
- String day
- String period
Elective Course
- int seats
guardians
Guardian
- String name
- String relation
- String phone
- Boolean emergency
+ addGuardian()
previous education
assigned Batch
EducationRecord
- String institute
- String qualification
- String year
+ addEducationRecord()
Batch
- String name
- String year
+ checkseatavailability()
+ assignStudent()
+ transferStudent()
assessed by
Examination
- String exam Id
- String type
- Date date
- Int MaxMarks
+ schedule()
+ assignEvaluationMethod()
+ attachResult()
groups
Exam Group
- String name
+ add Exam()
analyzes
Report Center
- String report
- String chart
+ generate Report
+ generate Chart
uses
GPA Evaluation
- String scale
CCE Evaluation
- String indicators
CWA Evaluation
- String weights
Evaluation Method
- String methodName
+ evaluate()
results
Exam Results
- String resultId
- String studentId
- double marks
- String grade
+ publish()

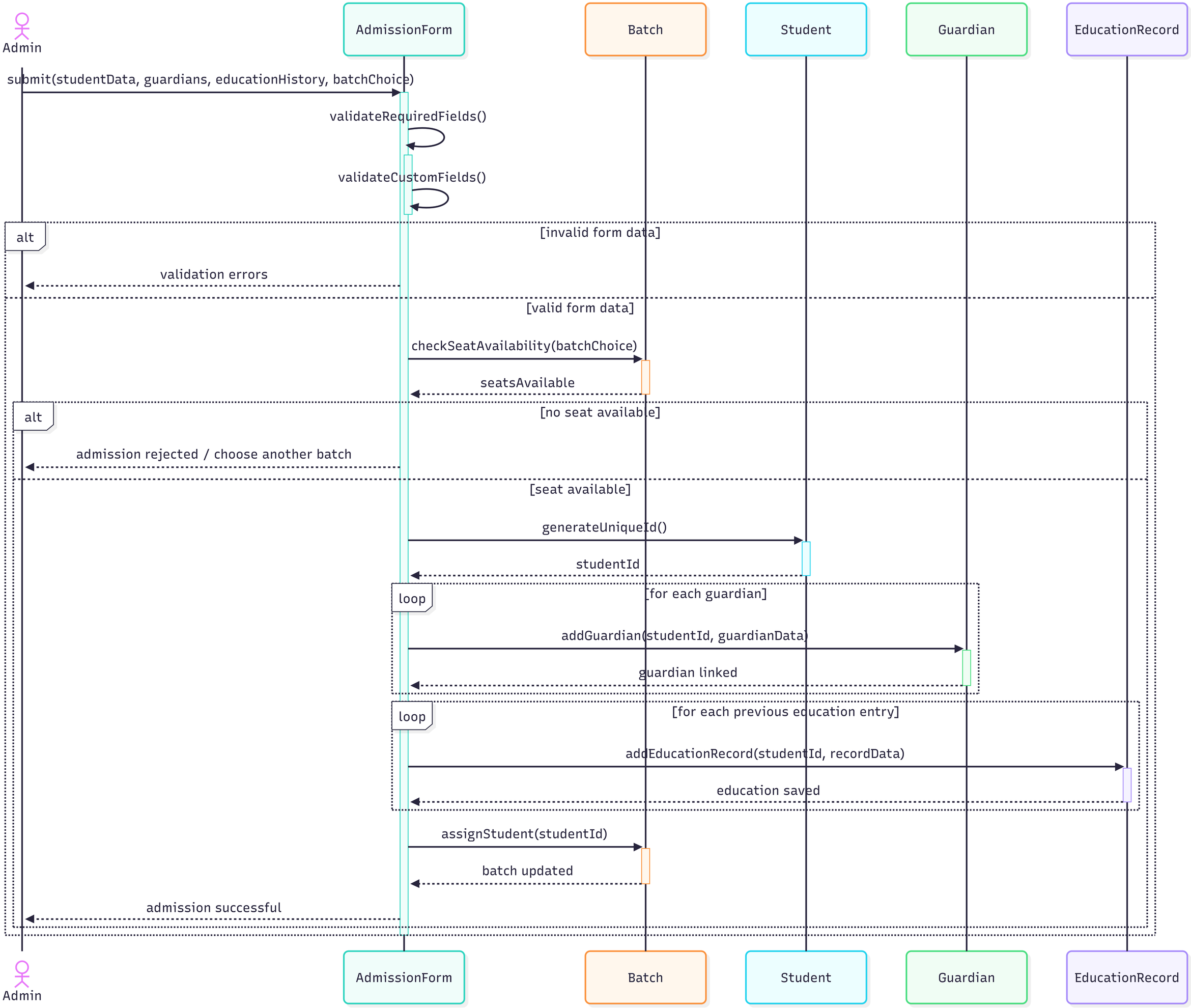
3.


Payroll form
- String formId
- String status
+ submit()
+ create Payslip()
Employee form
- String formId
- String status
+ submit()
1
fields
*
1
fields
*
Form Field Ref
+ reference: Page 2
Defaulter Report
- Date generatedOn
+ generate()
flags
Student Ref
+ reference: Page 2
*
paid by
1
checks unpaid
Fee Schedule
- String feeCategory
- double amount
- Date collectionDate
+ define()
1
settles
*
Fee Payment
- String paymentId
- double amount
- Date paidOn
+ process
Employee
- String employeeId
- String full Name
- String role
- String department
- String basicSalary
- Boolean Hr
+ generate EmployeeId()
+ getSalaryDetails()
+ get Payslips()
+ get Leave Applications()
1
payslips
*
Donations
- String donorName
Income
- String source
Asset
- String assetType
Payslip
- String month
- String status
- double basic Salary
- double allowances
- double deductions
+ calculate Net()
+ approve()
+ reject()
Liability
- String liability Type
Expense
- String category
records as
1
leave applications
*
*
login account
0..1
Leave Applications
- String leave Type
- Date start Date
- Date end Date
- String status
+ approve()
+ reject()
User Account Ref
+ reference: Page 1
Financial Record
- String record Type
- String category
- Date date
- double amount

Task 1.2: Responsibilities Table

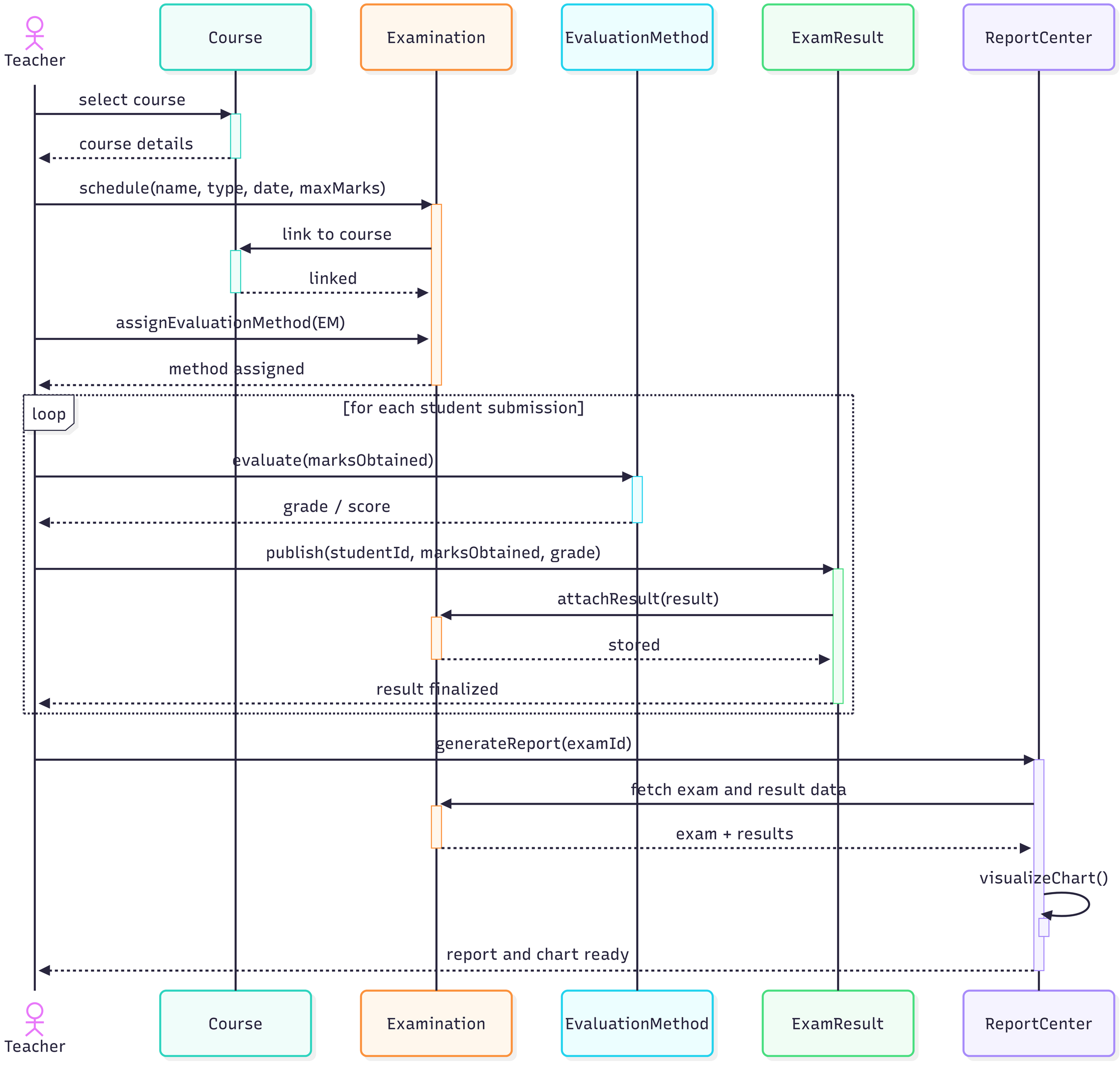
Class Name	Module	Primary Responsibility
Dashboard	Dashboard	Acts as the central navigational hub featuring an innovative search bar and displays the latest news on login.
SystemConfiguration	Dashboard	Manages global settings including language, country, currency, time zone, and grading systems.
SMSService	Dashboard	Integrates SMS functionalities for sending single or group alerts to users.
UserAccount	User Management	Manages login credentials and specific privileges based on assigned roles.
Role / Permission	User Management	Enables role-based access control to set standards based on organizational responsibility.
Student	Student Details	Core entity representing a student, holding a generated unique ID and graduation facility data.
Guardian / EducationRecord	Admission	Stores multiple guardians, emergency contact details, and previous education history for a student.
AdmissionForm	Admission	Represents fully customizable forms to meet specific school standards during student onboarding.
Course / ElectiveCourse	Courses	Represents educational programs and electives that students are enrolled in.
Examination / ExamGroup	Examinations	Creates and groups exams based on grades, marks, or custom types.
EvaluationMethod	Examinations	Defines the grading logic, supporting GPA, CCE, and CWA evaluation methods.
ReportCenter	Examinations	Generates automated, quick, and on-demand statistical and graphical reports for analytical insights.
AttendanceRecord	Attendance	Handles the easy marking of daily attendance with optional notes or remarks.
AttendanceReport	Attendance	Generates filterable attendance reports (daily, monthly) for quick analysis.
Timetable / ScheduleEntry	Time Table	Manages the drag-and-drop creation and scheduling of classes.
ScheduleAlert	Time Table	Generates real-time alerts on weekly class limits and employee workload constraints.
Employee	Human Resources	Manages end-to-end employee details from admission to exit.
EmployeeForm / PayrollForm	Human Resources	Provides customizable data entry forms for employee admission and payroll setup.
LeaveApplication	Human Resources	Drives the efficient leave management system for staff.
Payslip	Human Resources	Facilitates robust payroll management with authenticated one-click approval or rejection.
Message / BroadcastMessage	Messages	Handles inbuilt communication between users and broadcasts information regarding school events and news.
NewsItem / Comment	Manage News	Allows creation of news using rich text format (RTF) and manages user comments.
FeeSchedule	Finance	Defines comprehensive fee classifications and separate fee collection dates.
DefaulterReport	Finance	Analyzes and identifies fee defaulters.
FinancialRecord	Finance	Base entity to track all financial movements within the institute.
Income / Donation / FeePayment	Finance	Tracks incoming funds, processing instant fee submissions and donations.
Asset / Liability / Expense	Finance	Categorizes and tracks the institute's assets, liabilities, and outgoing expenses (like payroll).

Task 1.3
Sequence 1: Student Admission

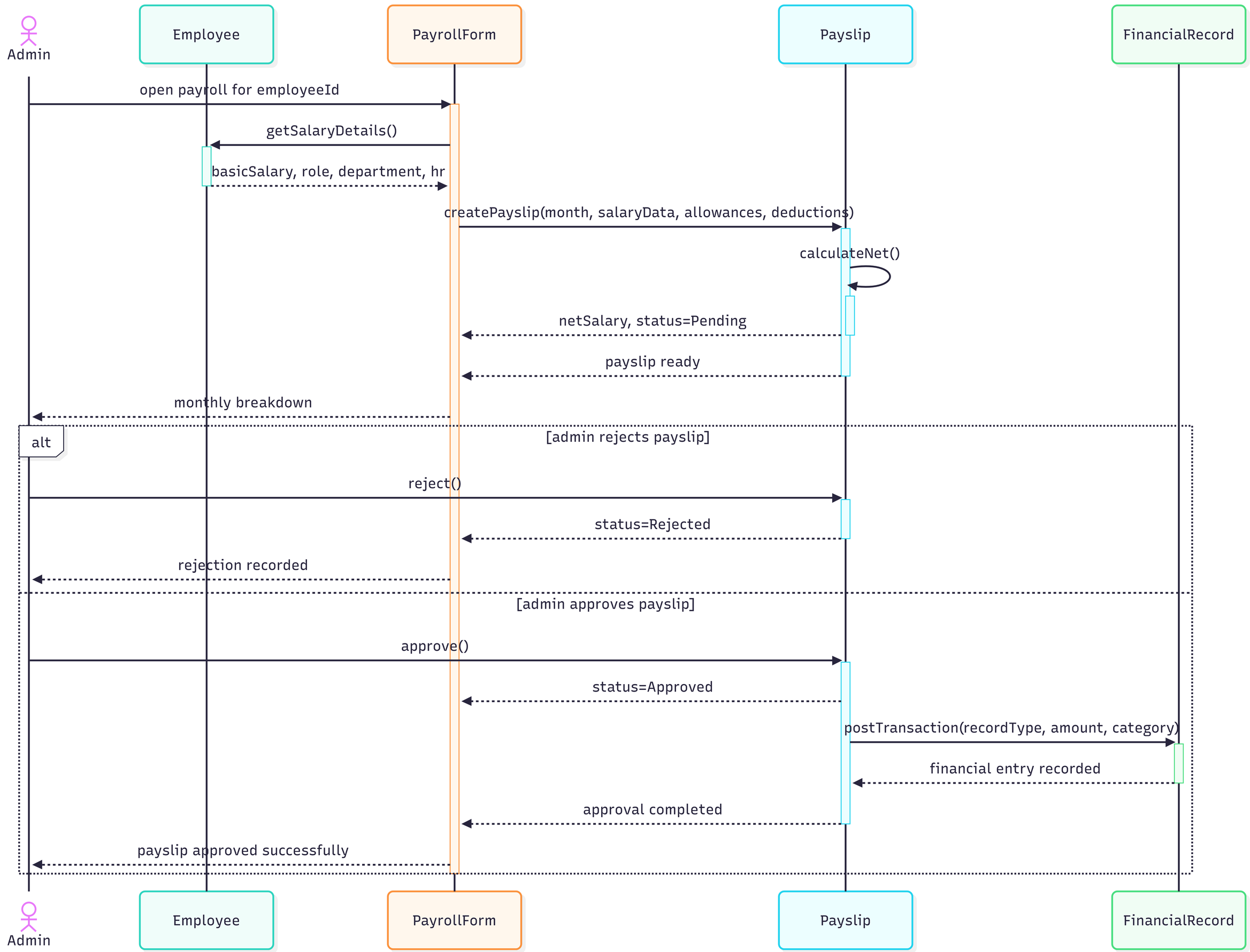


Task 1.3

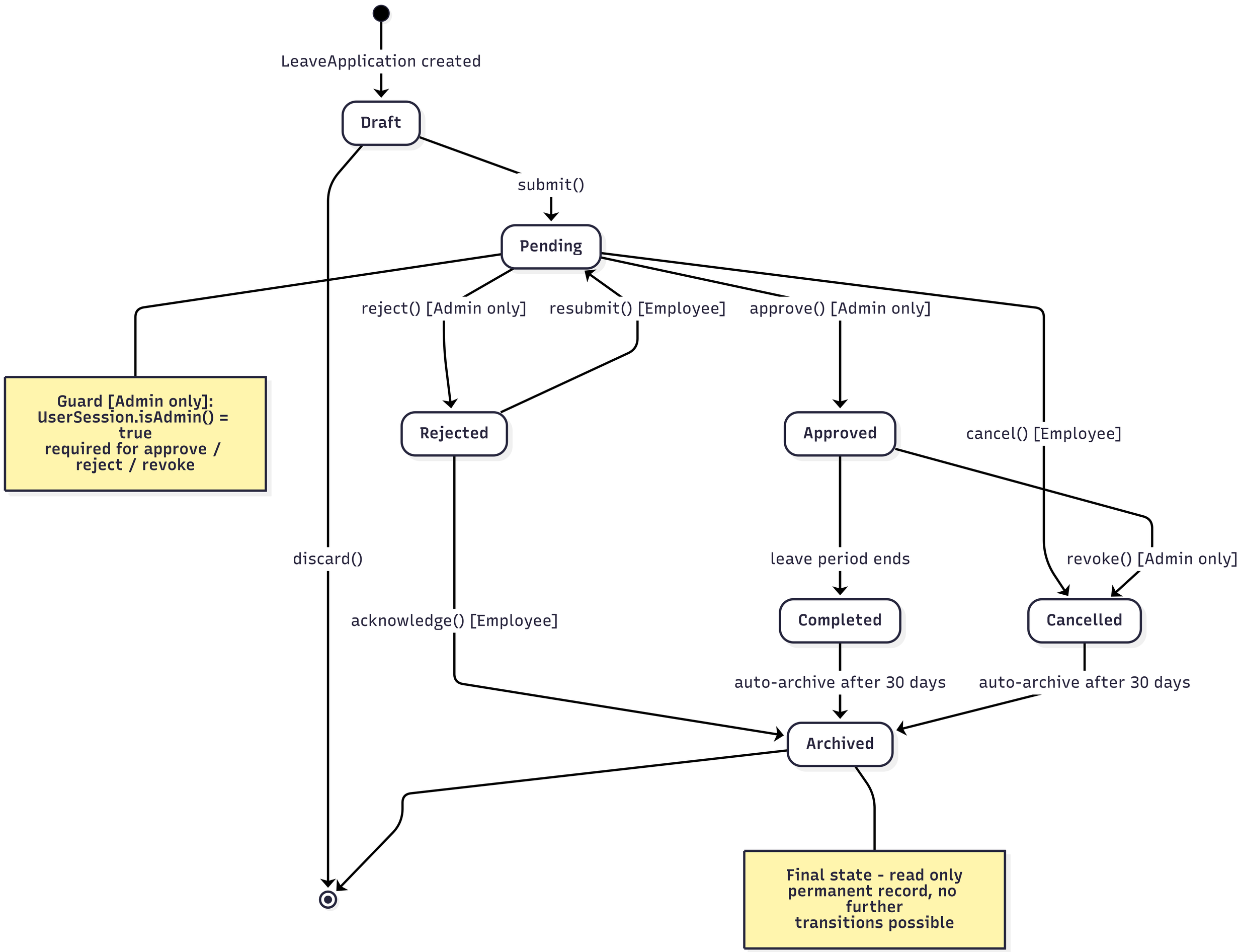
Sequence 2: Examination Evaluation Process



Task 1.3
Sequence 3: Payroll Management Process



Task 1.3:
State Diagram for Leave Request



TASK 1.4:

DESIGN NARRATIVE AND ANALYSIS

The main design goal was to represent the IMS as a set of collaborating domain areas, each responsible for a clear part of institute operations.

A major design decision was to keep the **Dashboard** as a navigational and coordination class rather than letting it absorb the responsibilities of every module. It is central from a user point of view, but not central from a domain-logic point of view. By limiting the dashboard to navigation, display, and configuration-oriented behavior, the design avoids turning it into a “god object” that knows too much and controls too much.

- *Low Coupling:* The design also attempts to maintain low coupling between modules. Student admission, examinations, HR, and finance are related, but they are not tightly fused together.
  For example:
  The admission process needs to know about Student, Guardian, EducationRecord, and Batch, but it does not need to know the internals of payroll, finance reporting, or messaging. Likewise, payroll approval touches Payslip, Employee, and FinancialRecord, but it does not depend on the internal structure of Course, AttendanceRecord, or NewsItem. This loose inter-module linkage is intentional.
- *High Cohesion:* The system also aims for high cohesion by assigning each major class a focused responsibility (i.e. each class is highly focused with a well defined purpose).
  For example:
  Examination, ExamResult, EvaluationMethod (and also the separate methods for evaluation, GPA, CCE and CWE) have been separated so that it is flexible and each class has its own functionality.
  On the HR side, Employee, Payslip, PayrollForm, and LeaveApplication have similarly focused responsibilities. In finance, FinancialRecord is used to unify transaction-like records rather than scattering fee, payroll, and other financial effects into multiple disconnected classes. This gives each class a clear reason to exist and a clear reason to change.
- *Separation of Concerns:* The system distinguishes between platform concerns, domain concerns, and cross-cutting concerns. Platform concerns include authentication/session-like behavior, roles/permissions, dashboard navigation, and configuration. Domain concerns include student lifecycle, examination management, timetable scheduling, and employee/payroll management. Cross-cutting concerns include messaging, news distribution, and finance record posting. This separation prevents classes from mixing unrelated responsibilities.

- *Law of Demeter:* The design achieves this by trying to avoid long chains of object navigation and hidden dependency paths. Instead of allowing any class to directly reach deep into others, interactions are mediated through the most relevant collaborator.
  For example: admission-related behavior is initiated through AdmissionForm, which then collaborates with the Student class to get any other details as needed.
- From a maintainability perspective, the design supports change reasonably well. If the institute introduces a new evaluation policy, the EvaluationMethod branch can be extended without redesigning the entire exam module. This is one piece of the evidence that proves the design prefers extension through focused classes rather than global changes across the entire model.

TASK 1.5:

DESIGN PATTERNS:

1. *Strategy Pattern in Examination Evaluation:*
   Its core idea is “define a family of algorithms, encapsulate each one, and make them interchangeable”.
   This pattern fits the examination domain very well. Different institutes or departments may use different methods of evaluating performance. If the logic for GPA, CCE, and CWA were all placed directly inside Examination, then the class would become bloated and would require modification every time a new method was introduced. By contrast, with an EvaluationMethod abstraction, Examination can simply collaborate with whichever evaluation method is assigned. This improves extensibility and supports the open/closed principle: new evaluation types can be added without rewriting the Examination class itself.

2. *Role-Based Access Control as a Policy/Authorization Pattern:*
   The design uses UserAccount, Role, and Permission to enforce role-based access control. The assignment explicitly requires role-based privileges and standards based on organizational responsibility. Instead of storing scattered booleans or embedding authorization logic in every domain class, the design centralizes access control through user roles and permissions.
   This resembles a policy-based design structure. Role groups privileges, Permission expresses specific allowed actions, and UserAccount is linked to the appropriate role. The advantage is that security and authorization are treated as a distinct concern. For example, approving payslips or moderating news should be allowed only for certain roles. With this design, access rules do not have to be reimplemented in every feature area separately. This reduces duplication and makes the system easier to manage if responsibilities change.

DESIGN ANTI PATTERNS AVOIDED:

1. *God Object/ Blob:* This is where one class accumulates too many responsibilities and becomes the center of all logic.
   In an IMS, the most obvious danger is the dashboard becoming such an object. Because the dashboard is the user's entry point and touches many modules, it would be easy to let it manage admissions, examinations, payroll, news, messaging, and configuration directly. The design resists this. Dashboard is kept as a navigational and coordinating class as discussed above.
   A similar anti-pattern risk exists in classes such as Student or Employee. These entities could easily become overloaded with everything related to academics or HR. Instead, the design pushes supporting responsibilities to surrounding classes.
2. *Excessive Cross-Module Coupling:*
   The design avoids a network of uncontrolled dependencies between modules. In this system, there is a strong temptation to connect everything directly to everything else: student data to payroll, payroll to messaging, dashboard to all internal details, exams directly to attendance reports, and so on. That kind of dependency web makes the design hard to understand and almost impossible to change safely.

TASK 1.6:

REFLECTION:

Two strongest aspects of the design

- The first major strength of the design is its modular structure. The system is modeled as an organized set of related areas: dashboard/platform, academic/student management, and HR/finance. This makes the design easier to understand. It also reflects the real structure of an institute, where admissions, examinations, payroll, messaging, and configuration are separate but coordinated functions. This modularity was followed because the assignment required conceptual coverage of the entire IMS,which is very interconnected and complex and needs to be structured. The final clustered design handles that challenge reasonably well.
- The second major strength is the reasonable balance between simplicity and extensibility. The model is not so minimal that major domain ideas disappear, and it is not so abstract that it becomes impossible to draw. Classes such as EvaluationMethod, ReportCenter, Role, Permission, LeaveApplication, and FinancialRecord add useful structure. At the same time, not every UI or helper concept is turned into a standalone class. This balance makes the design practical, manageable and at the same time extensible. New evaluation methods, permission sets, or financial record types can be added without rewriting the whole model.

Two weakest aspects of the design

- The first weakness is that the design shows some inconsistency at the abstraction level. While making the diagram we felt that there was a thin blurry line between what belongs to the structural UML modeling and what belongs to implementation. For example, some interactions suggest screen classes, database objects, while the class diagram was intended to stay at the conceptual domain level. Although in the final UML design we tried to mostly resolve this, still some confusion lies in how some workflows are described. In a stronger design there should have been more strictly controlled clear rules about what belongs in the conceptual UML and what belongs only in sequence or implementation design.
- The second weakness is that some classes remain behaviorally thin compared with their importance. A few major entities like students have structured attributes but not methods which utilize all of them so they appear more like records than rich objects. For instance, certain validation, search, update, and transition behaviors could be represented more explicitly in classes such as Student, Employee, FeeSchedule, or NewsItem.

Overall, the design tries to decently capture the conceptual map of the whole system. It covers the major institutional functions. The strongest part of the work is not any single class, but the way the classes are grouped into understandable subsystems with controlled dependencies. The weakest part is that it sometimes sacrifices behavioral depth to help with readability. Even so, the design follows standard qualities like cohesion, coupling, and separation of concerns.